

Digitalteknik, fortsättningskurs

2007

Föreläsning 15

Design för test

Varför testa?

Felaktiga kretskort/kretsar

Felaktiga kort	Levererade system	Kostnad	Felet upptäcks vid testning av kretskortet
5%	5000	\$1million	
1%	1000	\$200,000	
0.1%	100	\$20,000	
0.01%	10	\$2,000	

HW-debug

SW-debug

Produktion

Sluttest

Lverans

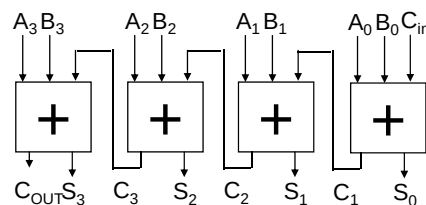
Fälttest

Felaktiga kort	Levererade system	Kostnad	Felet upptäcks hos kund
5%	5000	\$5million	
1%	1000	\$1milijon	
0.1%	100	\$100,000	
0.01%	10	\$10,000	

Hur ökar man testbarheten hos kombinatoriska nät

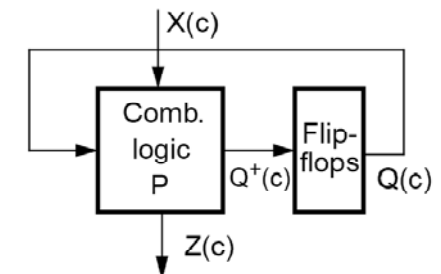
- Huvudproblemet
 - Kontrollerbarhet – få ingångar
 - Observerbarhet – få utgångar
 - Detekterbarhet – hur väl kan vi detektera felen

Ex ripple carry adderare



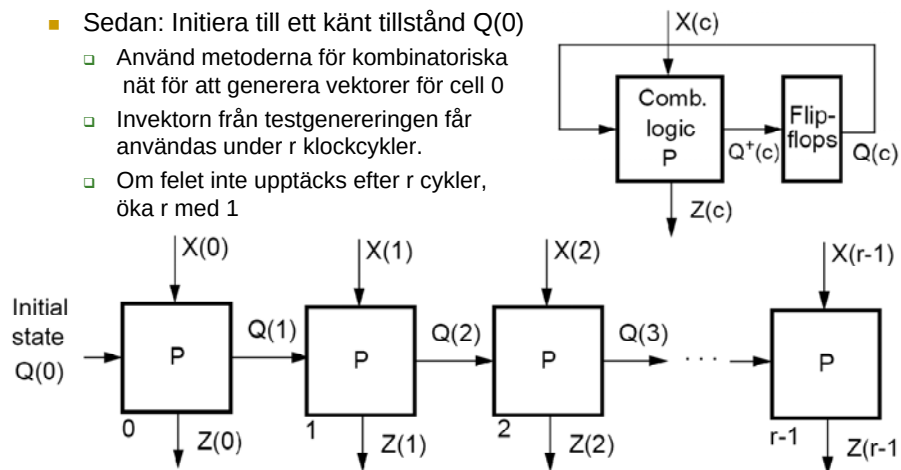
Testning av sekvensnät

- Svårare än kombinatoriska nät
 - Utsignalen definieras inte av insignalerna
 - P G A minnet
- Följande aspekter måste beaktas
 - Minneselementen måste sättas i ett känt tillstånd
 - Både invektorn och tillståndet måste ingå i testvektorn.
 - Mer än en invektor kan behövas för att aktivera ett fel



Generering av test för sekvensnät

- Vanligas: översätt sekvensnätet till ett iterativt kombinatorisk nät.
- Sedan: Initiera till ett känt tillstånd $Q(0)$
 - Använd metoderna för kombinatoriska nät för att generera vektorer för cell 0
 - Invektorn från testgenereringen får användas under r klockcykler.
 - Om felet inte upptäcks efter r cykler, öka r med 1



Konstruktion för testbarhet

- Ett stort problem när man testar är att observerbarheten och kontrollerbarheten hos kretsar ofta är relativt dålig, det är svårt att aktivera och propagera fel.
 - För att förbättra testbarheten använder man sig av olika design-for-test-metoder.
 - Nackdelen med DFT är ökad komplexitet, mer kiselåtgång, och viss prestandaminskning.

Konstruktion för testbarhet

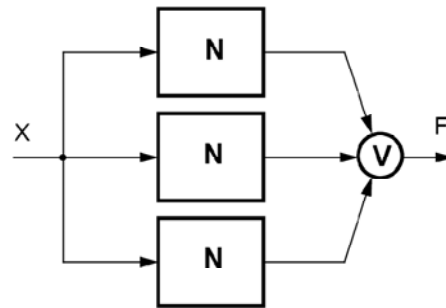
- Två möjliga angreppssätt
 - Ad hoc metoder
 - kan användas i efterhand
 - Strukturerade konstruktionsmetoder
 - Testbarhet finns med redan från början

Ad hoc metoder för ökad testbarhet

- Välkända tekniker baserat på erfarenhet för att öka styrbarhet och observerbarhet
 - Lägg till testpunkter.
 - Genom att lägga till extra ledare till/från kretsens utsida kan man öka dess observerbarhet och kontrollerbarhet
 - Initiering – reset
 - Få kretsen till ett välkänt tillstånd
 - Oscillatorer och klockor
 - Koppla bort den ordinarie under test
 - Partitionera stora kretsar/system
 - Det är mycket lättare att generera test, samt testa små kretsar än att testa stora. Genom att partitionera rätt kan man dessutom i vissa fall testa delar av kretsen parallellt, vilket ökar hastigheten.

“ad hoc” -metoder (forts)

- Redundant logik
 - UNDIK!
 - Inga TMR
- Globala återkopplingar
 - Bryt återkopplingen med logik
- Asynkrona sekvensnät
 - UNDIK



(a) TMR design

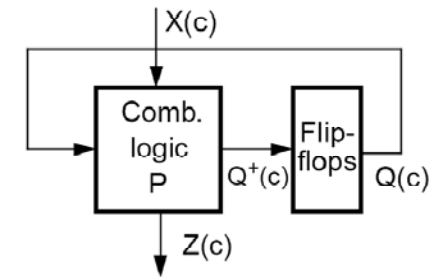
Strukturerad design för test

- Sekvensnät är besvärliga så delat upp dom i:

- Kombinatorisk del
- Register

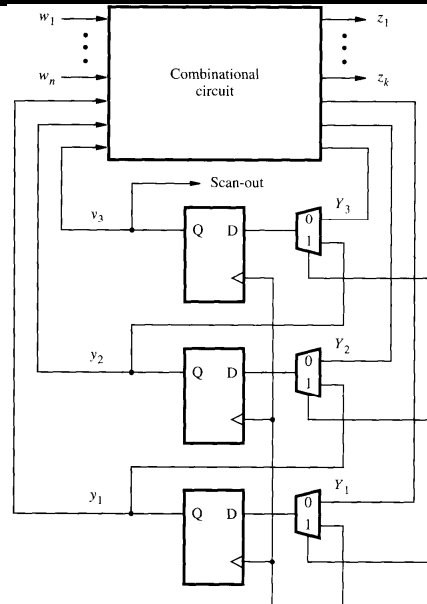
- Använder algoritmerna för kombinatoriska nät på den kombinatoriska delen

- Hanterar registren separat



Skannings kedjor

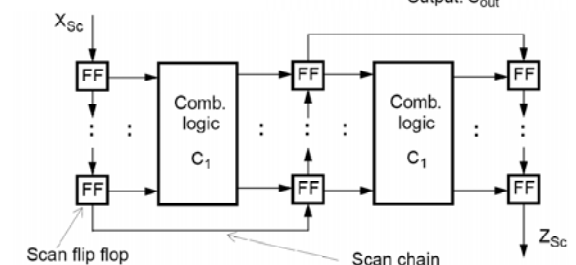
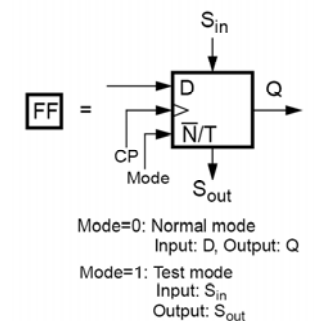
- Scan-kedjor är den vanligaste DFT-metoden. Samtliga minneselement i kretsen kedjas ihop till ett shiftregister så att man lätt kan klocka in/ut data i kretsen.
- Kostar ca 30% jämfört med vanliga vippor
- Skiljer mellan full och partiell skanningstekniker



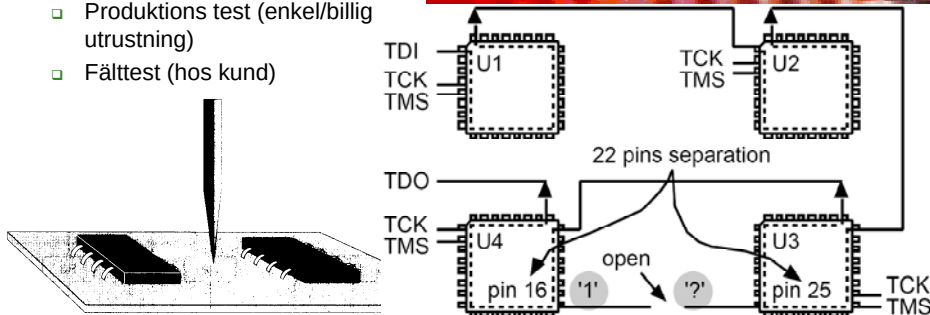
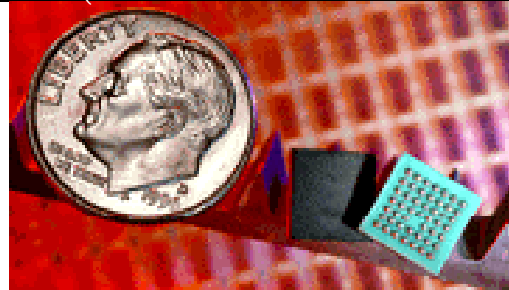
Skannings kedjor (forts)

- Samtidigt som en vektor skiftas in skiftas resultatet ut.

1. Mod test
2. Skifta in testvektor – ut resultat
3. Mod normal
4. Klockpuls
5. Börja om ned nästa vektor (1)

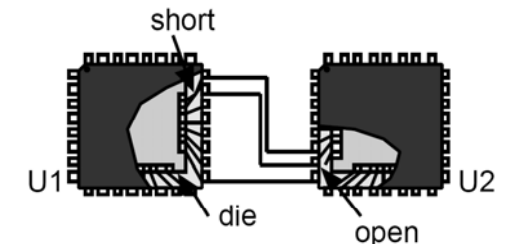
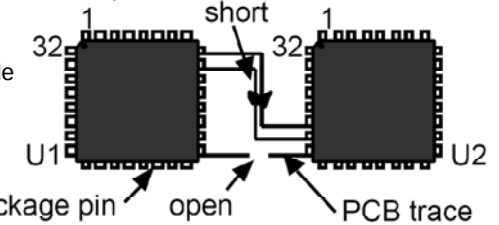


- Den vanligaste metoden för skannings kedjor
- Ger möjlighet till testning
 - Av alla kretsar (fullständig)
 - Av kretskortet (PCB)
 - Produktions test (enkel/billig utrustning)
 - Fälttest (hos kund)



Boundary-Scan (IEEE 1149.1) (forts)

- Boundary-Scan löser problemet:
 - att testa kretsar som är monterade annars
 - kan förstöra kretsen
 - BGA- kapslar
 - att testa kretskort
 - kan bestå av 20 ledningslager (eller mer) dolda via
 - densiteten är hög

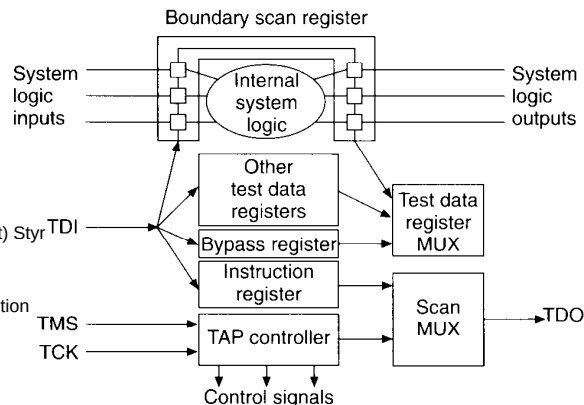


Boundary-Scan (IEEE 1149.1) (forts)

- Kallas även JTAG
- Elementet i arkitekturen

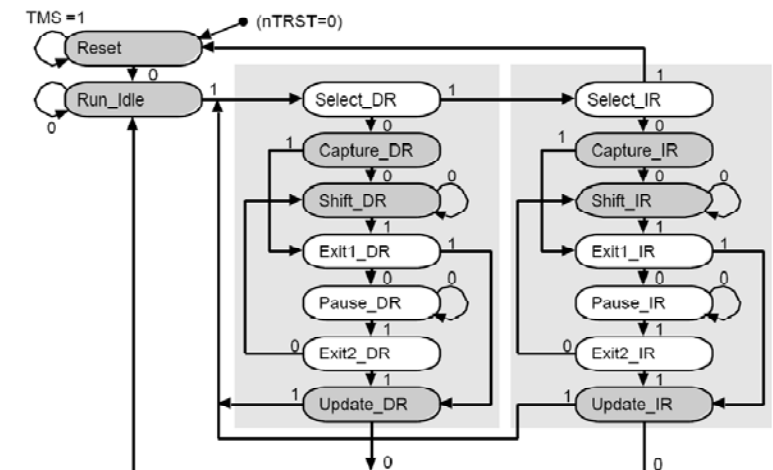
1. Test access port (TAP)

- Består av 5 pinnar
 1. TDI (Test data in)
 2. TDO (Test Data Out)
 3. TMS (Test Mode Select) Styr FSM
 4. TCK – Test klockan
 5. TRST (Test Reset) - option



JTAG (forts)

2. TAP styrenhet insignalier TCK, TMS



JTAG (forts)

3. Test data register

Alla komponenter i skannkedjan måste vara ansluta med en speciell cell.

Möjliggör bypass av en komponent.

Andra register såsom IC (som kan innehålla en identifierare) kan läggas till.

Vi programmerar våra Xilinx kretsar i labbet med JTAG

4. Instruktionsregister: minst två bitar. Definierar användningen av [3]

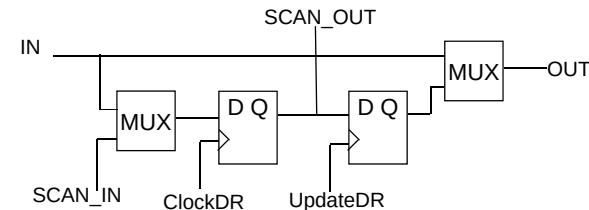
JTAG (forts)

■ Typisk Boundary-Scan cell visas nedan.

- För en in pinne ansluts IN till pinnen och OUT till kretsens logik (CORE).
- För en pinne som används som utgång: CORE→IN, OUT→pinne

■ Cellen har 4 moder

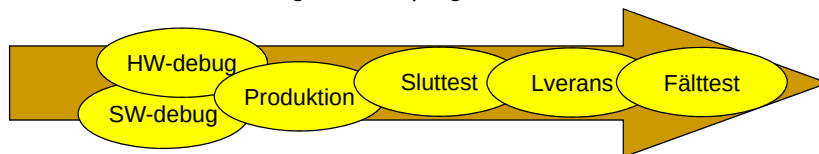
1. Normal mod: cellen är fränkopplad.
2. Scan mode: testvektorer skiftas in, resultat ut
3. Capture mode: cellen fångar resultatet av simulering
4. Update mode: data flyttas från först vippan till den andra.



Användningsområden - JTAG (forts)

■ Kan kraftigt reducera kostnaderna

- Eftersikt stöd för avlusning av både program och maskinvara



□ Fördelar

- Underlättar funktionstest
 - Minskar behovet av nåldyna
- Förenklar testprogrammering
- Kräver endast ca 2-5% extra chipsyta

□ Nackdelar

- Signalfördröjning – muxar
- 4 pinnar + ledningar
- De flesta kretsar måste ha JTAG
- ~Lång testtid
- Endast digitala delar

Inbyggd självtest

■ Är en teknik för att låta kretsen testa sig själv

- I produktion – ingen testutrustning behövs
- I fält – lätt att testa funktionen

■ Testningen sker när systemet inte är i drift

- Kan ej utföras under normal drift

■ Fördelar

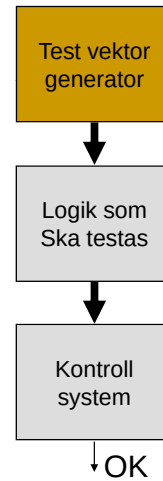
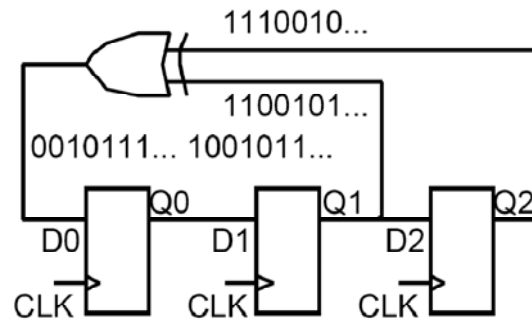
- Enklare eller inget testsystem
- Enklare testplanering
- Test vid normal frekvens
- Kretsbibliotek finns
- Många testvektorer ,enkelt

■ Nackdelar

- Stöd saknas i CAD/EDA- verktyg
- Prestanda i konstruktionen
- Extra komponenter behövs
 - Mer kisel yta
 - Fler ställen där det kan bli fel

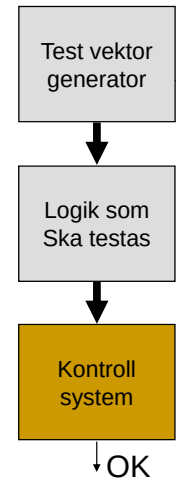
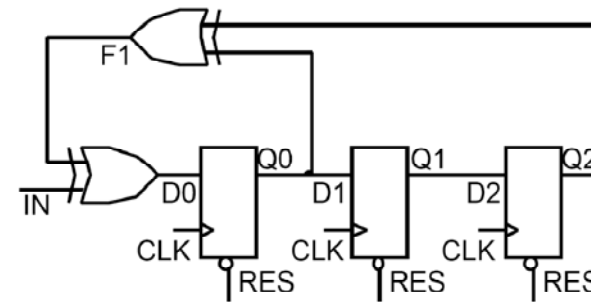
Inbyggd självtest *(forts)*

- Principen
- Hur?
 - "Slumptals generator"
 - ROM
 - Räknnare mm

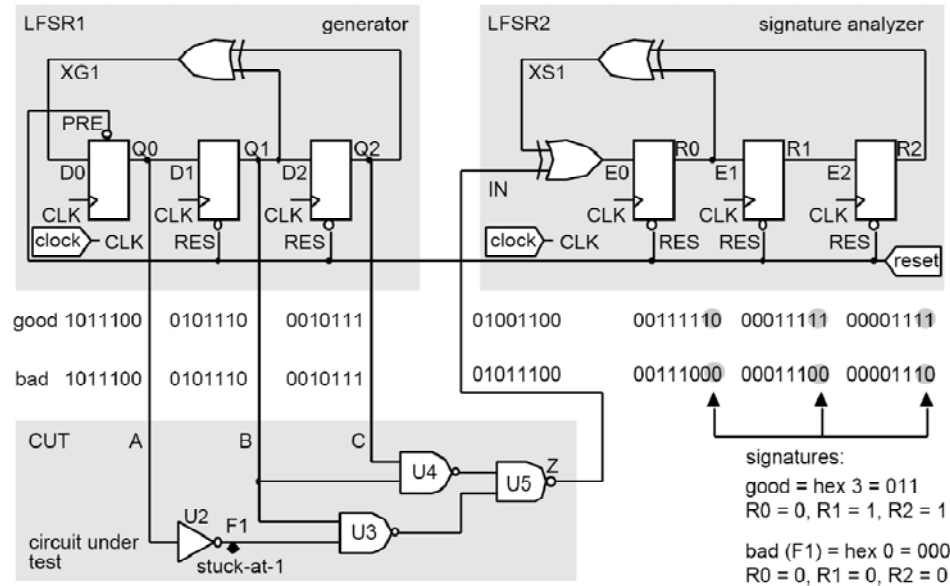


Inbyggd självtest *(forts)*

- Komprimering
 - ger oss en signatur

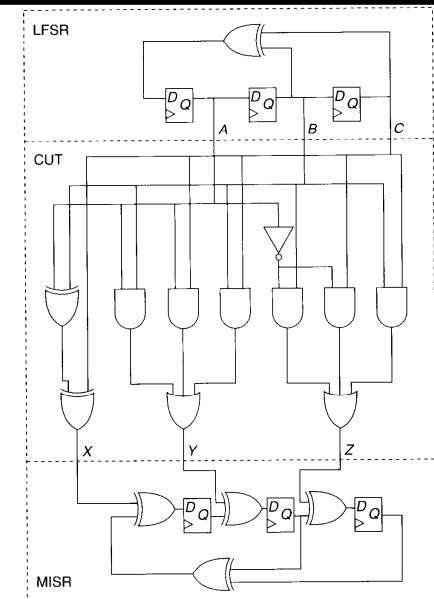


Exempel



Inbyggd självtest *(forts)*

- Flera utgångar



Mönsterkort *(forts)*

- Överhörning
 - långa pararalla ledare
→kapacitiv koppling
- Brus från spänningsmatning
 - avkoppling
- Reflektioner och termineringar
 - transmissionsledning



Testning av mönsterkort

- Spänningssätt
 - i värsta fall kan kretsar förstöras (se till att ha rätt spänning)
 - kontrollera om någon krets blir het
 - kontrollera att rätt spänningar finns till kretsarna
- Reset
 - kontrollera att rätt tillstånd erhålls
- Låg nivå funktions test
 - kontrollera att styrsignalerna beter sig korrekt
 - använd söndra och härska (divide-and-conquer) strategin
 - enkla funktioner förs
 - Därefter mer komplexa

Felsökning *(forts)* - Fullständig funktionstestning

Nu måste man verifiera funktionen hos hela systemet på mönsterkortet

Det är svårt och extremt tidskrävande att simulera ett helt system så det lär finnas fel kvar att upptäcka.

- Typiska fel
 - tillverkningsfel såsom
 - felaktig ledningsdraging, förstörda komponenter,
 - felaktig spänningsförsörjning
 - felaktiga specifikationer
 - felaktig information från datablad
- fel i gömda ledningslager är nästan omöjliga att åtgärda

Felsökning *(forts)*

- Timing – korrekt?
 - börja med en långsam klocka
 - viktigt att signalerna är stabila innan flanken kommer
- Tillförlitlighet – bör fungera länge
 - finns tillräckliga timing marginaler
 - temperaturtålighet

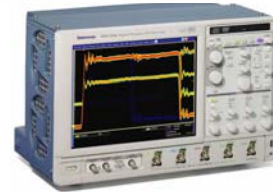
Instrument



Oscilloskop



Logikanalysator



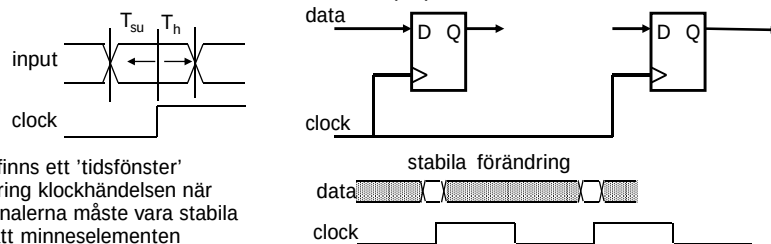
Metodik för timing

- Regler gör anslutning av komponenter till klocksignaler
 - garanterar att systemet fungerar om man följer dom strikt
- Olika angreppssätt beroende på vilka byggblock som används som minne
 - Fokuserar på system med flanktriggade vippor
 - finns i programmerbara logiska kretsar
 - många ASIC- implementeringar fokuserar på nivåkänsliga vippor
- Basregler för korrekt timing:
 - (1) korrekta insignaler, med avseende på tid, finns tillgängliga för vipporna
 - (2) inga vippor byter tillstånd mer än en gång per klockningshändelse

Metodik för timing (forts.)

■ Definition av termer

- Klocka: periodisk händelse, åstadkommer förändring av minnes-elementens tillstånd
kan vara stigande eller fallande flank eller hög/låg nivå
- setup time: minimal tid **före** klockhändelsen som insignalerna **måste** vara stabila (T_{su})
- hold time: minimal tid **efter** klockhändelsen som insignalerna **måste** vara stabila (T_h)

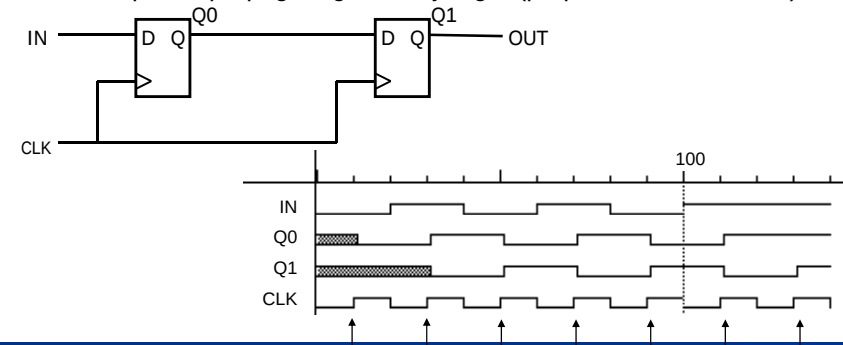


Det finns ett 'tidsfönster' omkring klockhändelsen när insignalerna måste vara stabila för att minneselementen ska få rätt värde

Kaskadkopplade flanktriggade vippor

■ Shift register

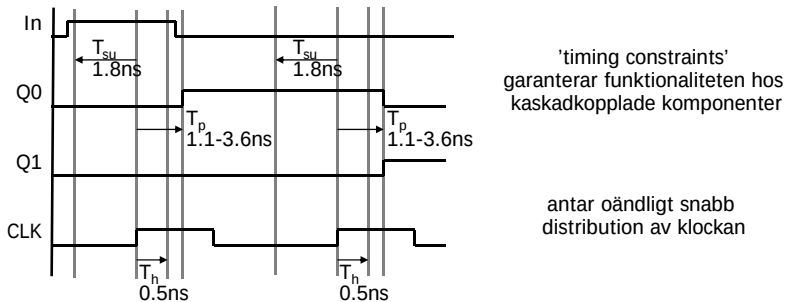
- nya värden går in i första steget
- medans förgående värde går till nästa steg
- setup/hold/propagerings fördröjningar (prop måste vara > hold)



Kaskadkopplade flanktriggade vippor (forts.)

Varför detta fungerar

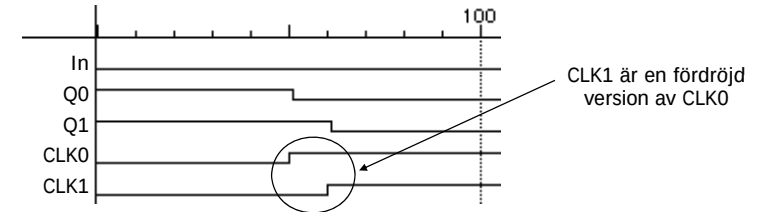
- Propageringstiden är större än 'hold' tiden
- Klockans bredd är större än 'setup' tiden
- Detta garanterar att det efterföljande steget samplar värdet innan det ändras till ett nytt



Skevhets hos klockan (Clock skew)

Problem

- korrekt beteende förutsätter att nästa tillstånd för alla minneselement bestäms av alla minneselement vid samma tidpunkt
- detta är svårt i hög prestanda system eftersom tiden det tar för klockan att anlända, är i samma storleksordning som fördröjningen genom logiken
- effekten av skevhets:



Original tillstånd: IN = 0, Q0 = 1, Q1 = 1

På grund av skevhets, blir nästa tillstånd: Q0 = 0, Q1 = 0, i stället för Q0 = 0, Q1 = 1

Metastabilitet och asynkrona signaler

Klockade synkrona kretsar

- signaler, tillstånd, och utsignaler samplas eller förändras i förhållande till en gemensam referens (klockan)
- både master/slave, flanktriggade

Asynkrona kretsar

- signaler, tillstånd, och utsignaler samplas eller förändras oberoende av gemensamma referenser (glitchar/ hasarder skapar problem)
- ex R-S latch

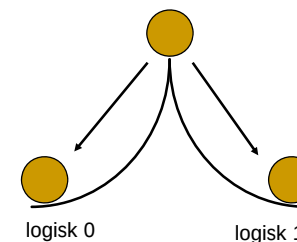
Asynkrona signaler till synkrona kretsar

- Signalerna kan byta värde när som helst, det går inte att garantera setup/hold tiderna
- farligt, synkrona signaler är att föredra
- Går dock inte att undvika (reset, memory wait, användare input mm)

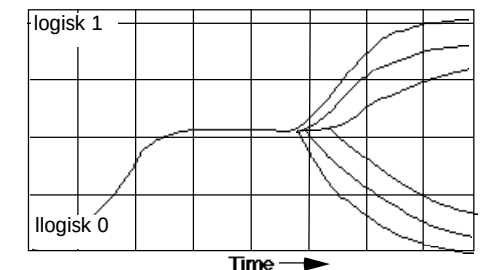
Synkroniseringsfel

Uppstår när vippans ingång ändras nära klockans flank

- vippan hamnar i ett metastabilt tillstånd – varken logisk 0 eller 1 –
- den kan stanna i detta tillstånd obegränsat länge
- detta är inte troligt i praktiken med det finns en liten sonorlighet för det



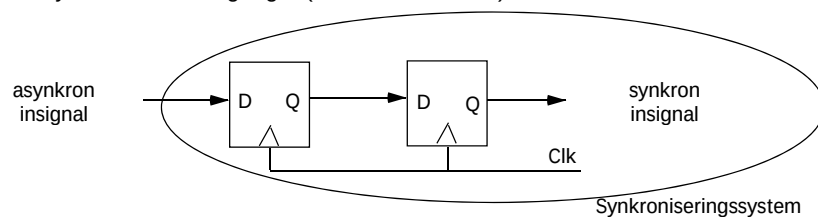
liten, men inte noll sonorlighet
att vippan fastnar
mellan sina tillstånd



Hur det kan se ut på ett oscilloskop
när synkroniseringsfel uppträder

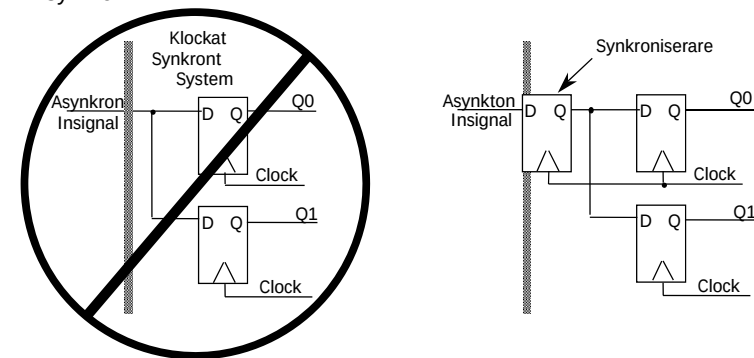
Att hantera synkroniseringsfel

- Sandorlikheten för fel funktion kan aldrig reduceras till 0, men den kan minskas radikalt
 - (1) använd långsammare system klocka
detta ger minneselementet mer tid för att nå ett stabilt tillstånd; synkroniseringsproblem är ett stort problem för snabba system
 - (2) använd snabbast möjliga logikteknologi för synkroniseringen
 - (3) kaskadkoppla två synkroniseringselement
synkroniserar två gånger (båda måste falera)



Hantera asynkrona insignaler

- Aldrig **aldrig aldrig** tillåt asynkrona insignaler att vara kopplade till mer än en vippa
 - synkronisera så snart som möjligt och behandla därefter signalen som synkron



Hantera asynkrona insignaler (forts.)

- Vad kan bli fel?
 - signalen ändras för nära klockflanken (villkoret för setup tid ej uppfyllt)

